



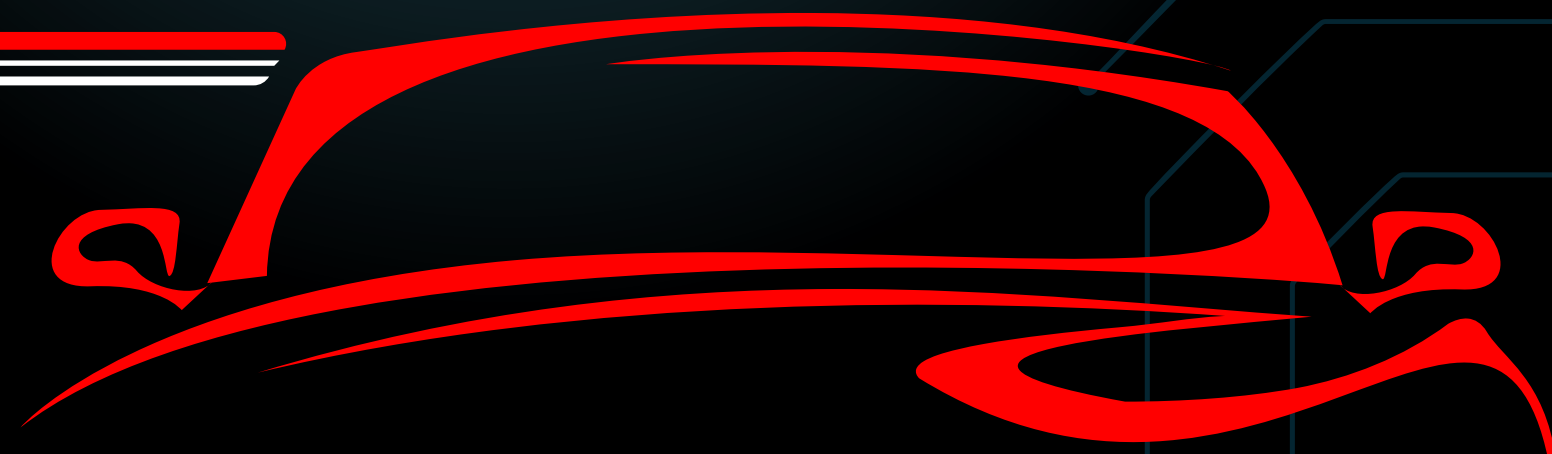
SMART PARKING SYSTEM



OUR TEAM

MARYAM ALAA

KHOLOD FAHMY



A smart parking management system uses IoT, sensors to simplify finding and managing parking . It detects available spaces and communicates this to drivers, reducing search time and traffic congestion. Operators gain tools like dynamic pricing and space These systems enhance convenience and help lower carbon emissions.

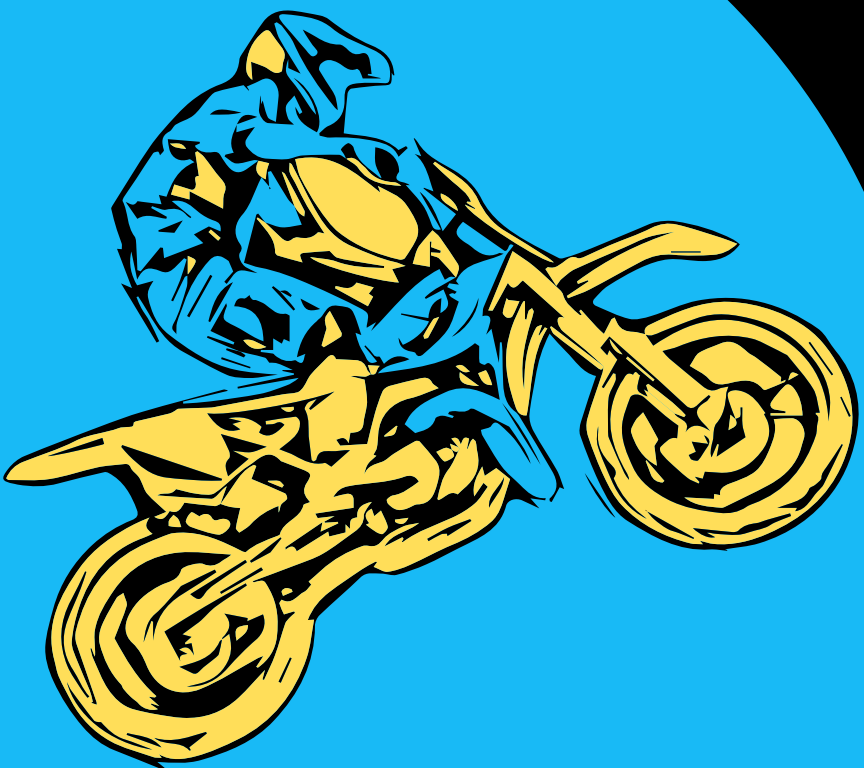
Menu-Driven Architecture: Simplified navigation loop for seamless operator workflow

ANSI Color Codes: Enhanced visual clarity using responsive terminal highlighting (Cyan/Green/Red)

Sanitized Selection: Strict runtime input traps to catch and reject invalid menu choices



CODE STRUCTURE



CODE STRUCTURE: DATA REPRESENTATION

- **Encapsulated Vehicle Record:** Custom struct to group all attributes into a single database unit.
- **Global Array State:** Manages the entire garage buffer through `v[MAX_SLOTS]` (200 slots)
- **State Tracking Flags:** Uses `.status` variable (1 for Occupied, 0 for Vacant) for visibility.

```
struct Vehicle {  
    int id;  
    char owner_name[50];  
    char plate_number[15];  
    char vehicle_type[20];  
    int slot_number;  
    int entry_hour;  
    int entry_min;  
    int status; // 1 = Occupied, 0 = Empty  
} v[MAX_SLOTS];
```



CODE STRUCTURE: DATA REPRESENTATION



Core Operations

- **add_vehicle():** Registers entry logs
- **remove_vehicle():** Triggers exit leanup
- **edit_vehicle():** Modifies profile fields securely
- **search_vehicle():** Finding vehicle by ID or Plate Number



Logic & Analytics

- **assign_slot():** Maps array indexes
- **release_slot():** Resets occupancy flags
- **calculate_fees():** Processes dynamic pricing



Data & Overview

- **print_vehicle_data():** prints data of vehicle on the system
- **display_overview():** displays garage overview and occupied percentage



Utilities & Helpers

- **find_vehicle_index():** Centralized ID lookup.
- **save_parking_data():** Writes database text files
- **load_parking_data():** Restores system memory state.

SYSTEM LOGiC



functions.h

Contains core system structures (struct Vehicle), configuration macros (MAX_SLOTS), and necessary library headers



functions.c

Implements the actual operational logic for functions like adding, removing, allocating slots, and calculating fees



main.c

Manages the main program execution loop, displays user CLI menus, and routes operations based on user input.

VEHICLE LIFECYCLE



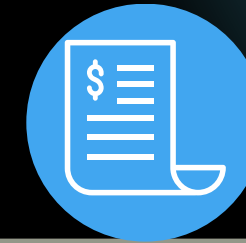
Step 1: Entry & Validation

`add_vehicle()` requests inputs and calls `find_vehicle_index()` to ensure the ID is unique before saving time logs



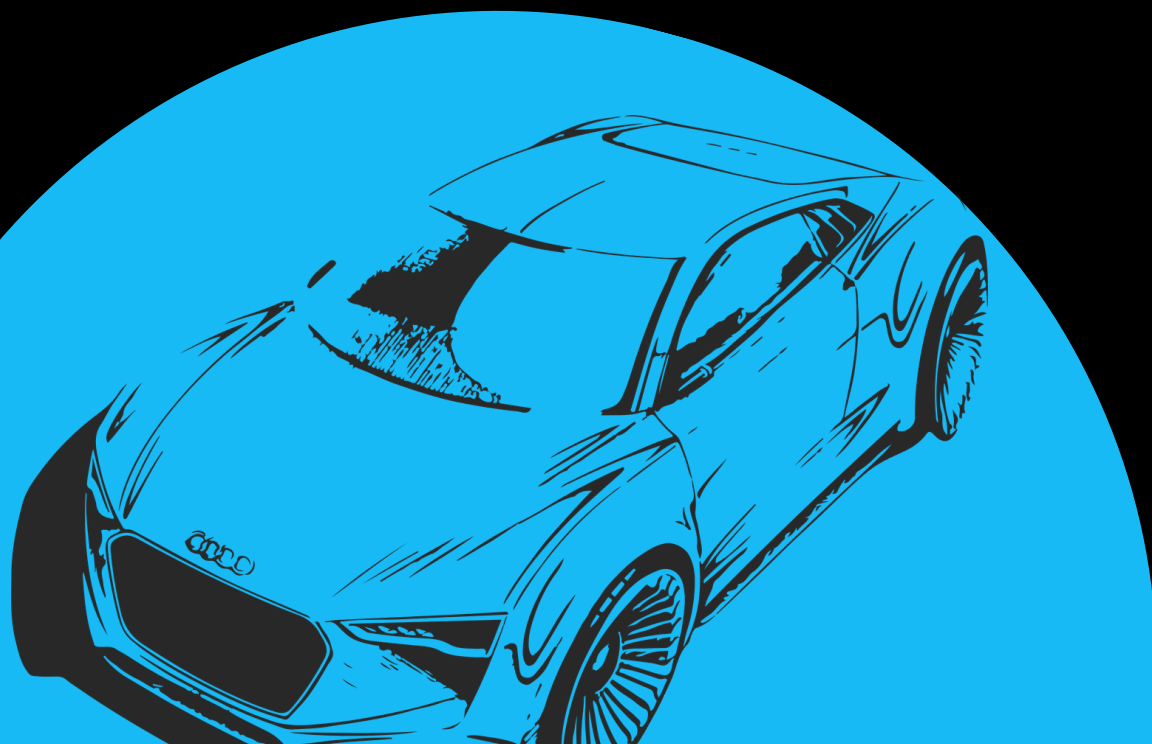
Step 2: Slot Allocation

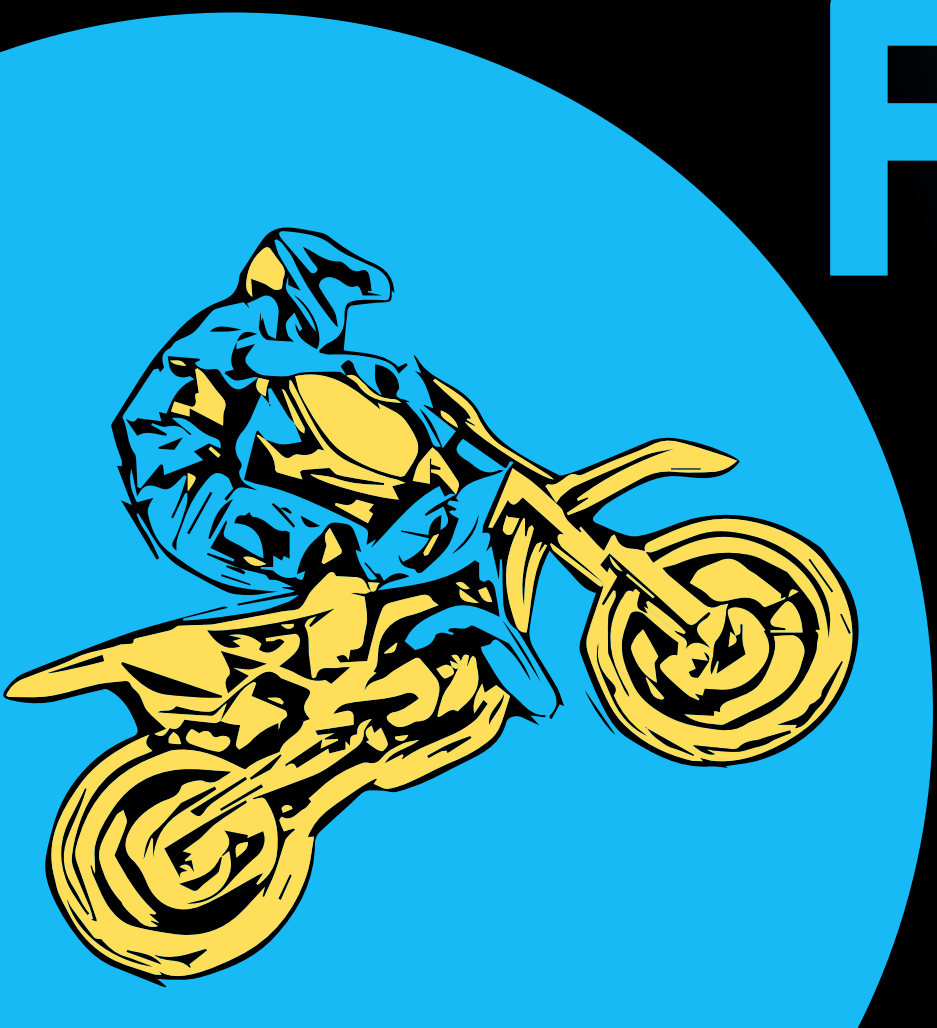
`assign_slot()` scans the global array, finds the first empty index [0-199], and locks `.status = 1`



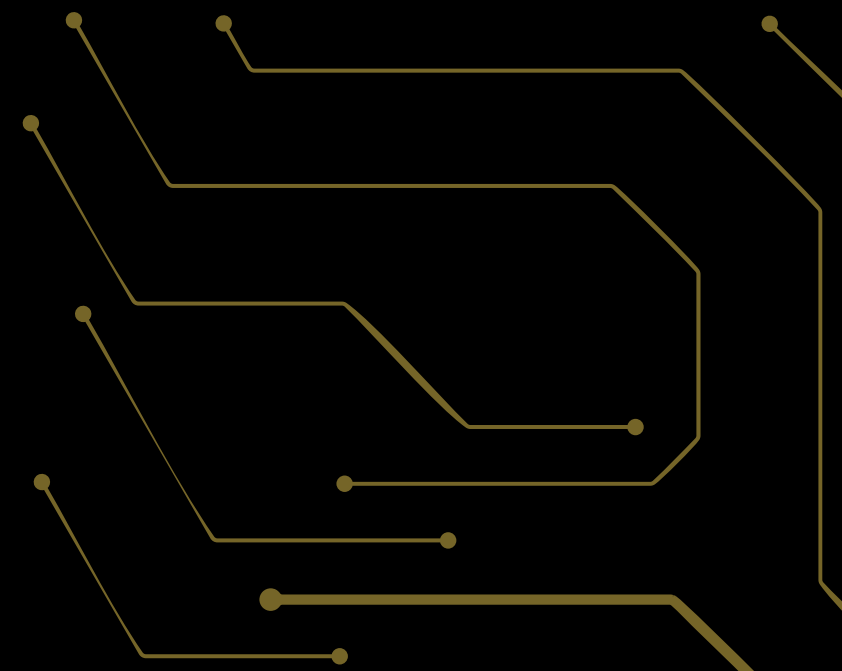
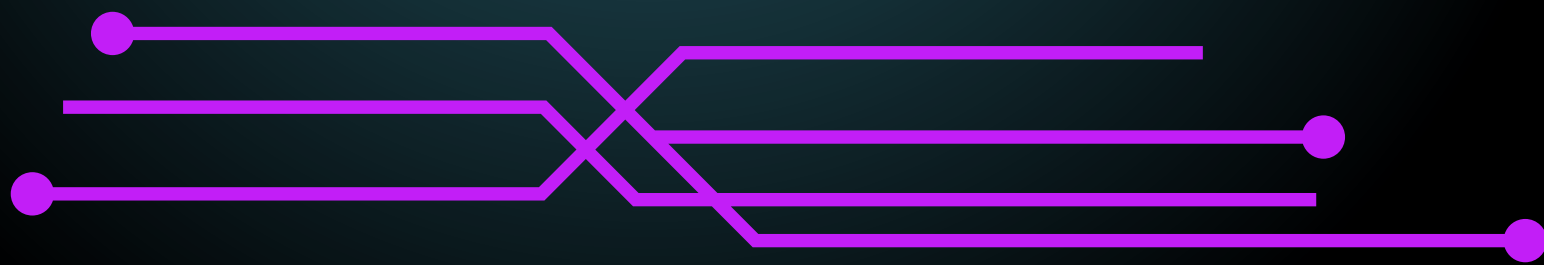
Step 3: Checkout

`release_slot()` calculates exact parking duration, evaluates fees using logic rules, and resets `.status = 0`.





KEY FUNCTIONALITIES



VEHICLE ENTRY VALIDATION

- ➔ **Capacity Check:** Verifies `vehicle_counter == MAX_SLOTS` to prevent garage overflow
- ➔ **Uniqueness Constraints:** Rejects duplicated IDs and Plate Numbers using helper functions `()` and `strcmp`
- ➔ **Input Sanitization:** Enforces valid time formats (Hours [0-23], Minutes [0-59]) via a `while(1)` loop
- ➔ **Slot Registration:** Locates the first vacant placeholder (`status == 0`) and calls `()assign_slot`

SLOT ALLOCATION

- ➔ **Manual Selection:** Users pick a slot [1-200], converted to 0-indexed format.
- ➔ **Range Validation:** Rejects any input outside the array boundaries
- ➔ **Occupancy Check:** Prevents overwriting slots that are already taken (`status == 1`).
- ➔ **Data Shifting:** Safely transfers vehicle payload to the `.chosen` memory index

VEHICLE EXIT & SLOT RELEASE

- ➔ **remove_vehicle():** Prompts user for ID and calls `find_vehicle_index()` to locate the car.
- ➔ **Fee Calculation:** Invokes `calculate_fees()` to compute and display the billing receipt.
- ➔ **release_slot():** Resets slot properties (`slot_number = 0`, `status = 0`) to mark it empty
- ➔ **Safe State Cleanup:** Decrements `vehicle_counter` and reloads the structural sub-menu

PARKING FEE CALCULATION

- ➔ **Automated Exit Time:** Uses `<time.h>` and `localtime()` to capture exact real-time exit hours and minutes
- ➔ **Overnight Correction:** Adds 1440 minutes (+24 hours) automatically if exit time spans into the next day.
- ➔ **Grace Threshold:** Implements a 10-minute grace period before rounding up to the next full billing hour
- ➔ **Tiered Pricing:** Charges a 10.0 EGP base rate for the first hour, plus 5.0 EGP for each additional hour.

DATA LOADING LOGIC

→ **load_parking_data():** Automatically restores all garage records from parking.txt at system startup

→ **Buffer Overflow Prevention:** Employs a strict boundary check (if $i \geq 200$) to protect system memory.

→ **State Recovery:** Synchronizes occupancy flags (status=1) and recalculates the global vehicle counter

DATA PERSISTENCE (SAVING DATA)

→ **save_parking_data():** Automatically serializes all active records to parking.txt upon exit or data modification

→ **Storage Optimization:** Filters through the database array to only write occupied slots (status == 1), avoiding corrupted or blank lines

→ **Data Serialization:** Formats data with custom pipe delimiters (|) to ensure seamless structure matching with the system loader

ERROR HANDLiNG



MEMORY & FILE GUARDS

Array Boundary Protection: Implements strict limits on manual slot allocation [1:200] to block memory out-of-bound crashes

Garage Capacity Overflow Guard: Automatically halts system inputs when vehicle_counter reaches maximum structural thresholds

File Pointer Integrity: Enforces NULL pointer checks during database startup and synchronization to prevent critical segmentation faults

```
// From assign_slot() & add_vehicle()
if (chosen_slot < 0 || chosen_slot > (MAX_SLOTS - 1)) {
    printf("Invalid Slot Number! Try Again.\n");
}
if (file == NULL) {
    return; // Safe exit on file corruption
}
```

DATA PROTECTION & VALIDATION

- **ID Collision Protection:** Cross-references input tokens dynamically using a centralized search helper (find_vehicle_index) to reject duplicated IDs during entry or editing.
- **License Plate Validation:** Enforces structural tracking uniqueness across all active garage entries via sequential string comparison (strcmp)
- **Sub-Menu Input Validation:** Features recursive recovery loops (default: blocks) to gracefully handle invalid operator keypad inputs without terminating the program

OPERATIONAL LOGIC CONTROLS

- **Overnight Rollover Correction:** Automatically compensates for the 24-hour clock cycle by adding a +1440 minutes offset when exit time falls before entry time
- **Redundant Release Guard:** Secures the database against double-clearance operations by safely rejecting checkout requests on already empty slots.
- **Precise Hourly Billing:** Implies the ceiling function (ceil) to ensure any fractional hour usage is fairly rounded up to the next tier rate

```
// [1] Midnight Rollover Fix
if (exit_total_mins < entry_total_mins) {
    exit_total_mins += 1440; // Adds 24 hours in minutes
}

// [2] Empty Slot Protection
if (current_slot == 0 || v[i].status == 0) {
    printf("Slot is empty or is already released!\n");
    return 0; // Safe exit flag
}
```

```
=====
...--> VEHICLE INFO SECTION <--...
=====

Enter ID ==> 2

FOUND SUCCESSFULLY :)

Vehicle ID ==> 2
Owner's Name ==> Maryam Alaa
Entry Time ==> 11:11
Slot Number ==> 2
Status ==> Parked

=====

Total Slots ==> 200
Occupied Slots ==> 3
Available Slots ==> 197
Occupancy Percentage ==> 1.50%

=====

[Any Key] Back To Menu |

=====

[1] View Another Vehicle    [Any Key] Back To Menu |
=====
```

```
=====
...--> ADDING SECTION <--...
=====

Enter the vehicle ID ==> 4
Enter The Owner Name ==> Maryam Alaa
Enter The Plate Number ==> wwe4
Enter the vehicle type ==> Car
Enter entry time (H:M) ==> 11:11

Choose A Slot [1:200] ==> 4

Successfully Assigned to Slot [4] :)

=====

[1] Add Another Vehicle    [Any Key] Back To Menu
=====
```

SYSTEM OUTPUT

```
=====
...--> REMOVING SECTION <--...
=====

Enter ID ==> 3

Car Parked For 12 Hours And 26 Minutes

TOTAL PARKING FEE ==> 70.00

VEHICLE REMOVED SUCCESSFULLY :)

SLOT [3] RELEASED SUCCESSFULLY :)

=====

Remove Another Vehicle    [Any Key] Back To Menu |
=====
```

```
=====
...--> SEARCHING SECTION <--...
=====

Searching by [1] ID or [2] Plate Number? ==> 1

Enter ID ==> 1

FOUND SUCCESSFULLY:)

Owner's Name : Maryam Alaa
Plate Number : wwe1
Vehicle Type : Car
Slot Number : 1
Entry Time : 11:11

=====

[1] Search Another Vehicle    [Any Key] Back To Menu
=====
```

```
=====
...--> EDITING SECTION <--...
=====

Enter ID ==> 3

FOUND SUCCESSFULLY :)

[1] Edit Owner Name    [2] Edit Plate Number

[3] Edit Vehicle Type

[4] Edit Vehicle ID    [5] Edit Entry Time

=====

Enter your Choice ==> 1

=====

Enter New Owner Name ==> |
=====
```

THANK YOU

IEEE – EMBEDDED SYSTEMS